

So far in this class, with the exception of k-Nearest Neighbors, we've focused mostly on *linear* models. In the regression setting, we've seen models that make predictions like $h(\mathbf{x}) = \mathbf{w}^\top \mathbf{x}$, and for binary classification we've seen models like $h(\mathbf{x}) = \text{sign}(\mathbf{w}^\top \mathbf{x})$ or ones that estimate the probability that $y = +1$. Linear regression models pick a vector $\mathbf{w}$ and say that the label is increasing in the direction of $\mathbf{w}$. Linear classification models pick a vector $\mathbf{w}$ and say that $\mathbf{w}$ points in the direction that the label becomes more likely to be positive.

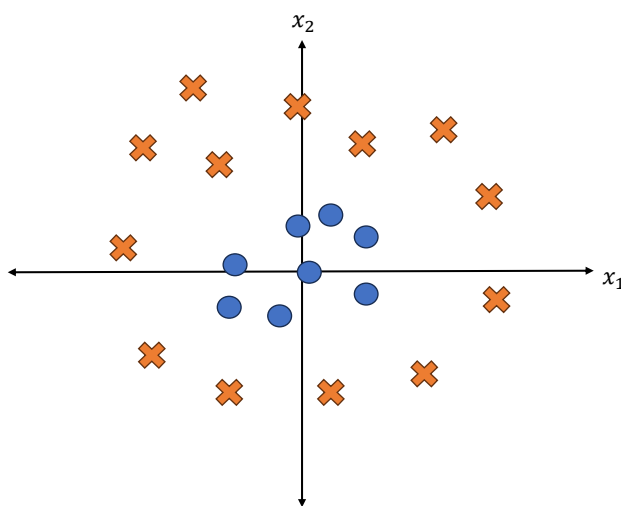


Figure 1: A linear model won't do much good here.

It's pretty easy to draw datasets for which these kinds of models will obviously fail. In the figure above, it's pretty clear that any line you draw is not going to do a good job of separating the blue circles from the orange crosses. In this lecture, we're going to leave linear models behind and start talking about *nonlinear* models.

1 Feature maps

One thing we discussed in class is the fact that linear models become quite a bit more powerful as they gain access to more features. Recall that we typically assume our data points are d dimensional real valued vectors, $\mathbf{x}_i \in \mathbb{R}^d$. As d increases, the dimensionality of $\mathbf{w}$ correspondingly increases, and models like $h(\mathbf{x}) = \mathbf{w}^\top \mathbf{x}$ become more expressive. Obviously, we can't always just add more features: sometimes, you're just already using all the data you have. One of the most fundamental strategy that we'll use to develop nonlinear models is to increase the dimensionality of the data by using the **existing** features to define **new** ones.

More formally, over the next several lectures we'll define *feature maps* $\phi(\mathbf{x}) : \mathbb{R}^d \rightarrow \mathbb{R}^D$ that map

input feature vectors $\mathbf{x}$ to higher $D > d$ dimensional feature vectors $\phi(\mathbf{x})$. Then, instead of learning models $h(\mathbf{x}) = \mathbf{w}^\top \mathbf{x}$, we'll learn models $h(\mathbf{x}) = \mathbf{w}^\top \phi(\mathbf{x})$. This latter model is still “linear” – but it's linear in the expanded feature space defined by $\phi(\mathbf{x})$, which can be nonlinear in the original feature space. This technique is the basis of both **kernel methods** and **neural networks**.

To make this more concrete, consider again the dataset in the figure above. Each point in the figure $\mathbf{x}_i$ is defined by two features—its position on the x_1 and x_2 axes respectively—and a label—whether the point is a circle ($y = -1$) or a cross ($y = +1$). Clearly, it's difficult to find a model of the form $h(\mathbf{x}) = w_1x_1 + w_2x_2$ for this dataset. However, suppose we define the feature map $\phi(\mathbf{x}) = [x_1, x_2, x_1^2 + x_2^2]$. In other words, ϕ takes one of these 2D feature vectors as input and adds a new, third dimension that just computes $x_1^2 + x_2^2$ from the first two already existing features.

With this new feature, it should be pretty clear that a linear classifier defined by $\mathbf{w}^* = [0, 0, 1]$ probably does pretty well. The new third feature essentially measures the (squared) distance of each point from the origin. As we get further from the origin, we start getting more crosses – the positive label. This model, $h(\mathbf{x}) = \mathbf{w}^{*\top} \phi(\mathbf{x})$ is still a “linear” model in the new three dimensional space, but it's clearly nonlinear in the original 2D space.

2 Going bigger: polynomial feature maps

The above example is a compelling story about how adding features can easily lead to nonlinear models. Adding a single, third feature to the toy data in our above example allowed us to define a very good model for this data by combining existing features. A natural question to ask is what happens if we continue playing this game of producing lots of combinations of features. Consider for example the following feature map that we'll call the polynomial feature map of degree 2:

$$\phi(\mathbf{x}) = \begin{bmatrix} 1 \\ x_1 \\ \vdots \\ x_d \\ x_1x_2 \\ \vdots \\ x_{d-1}x_d \\ x_1^2 \\ \vdots \\ x_d^2 \end{bmatrix}$$

This feature map contains the original features $\mathbf{x} = [x_1, \dots, x_d]$, but it also contains new features that consist of all pairs of feature products $x_i x_j$. In other words, for every pair of features in the original feature set, we add a new feature that is just the product of those two feature values. It's interesting to note that this feature map trivially solves the first figure's data as well: for $\mathbf{x} = [x_1, x_2]$, both x_1^2 and x_2^2 appear as features in $\phi(\mathbf{x})$. Putting some weight α on each of these new features results in a prediction that contains $\alpha x_1^2 + \alpha x_2^2 = \alpha(x_1^2 + x_2^2)$.

We call this a polynomial feature map of degree two because any degree two polynomial of the original input $\mathbf{x}$ can be written as $\mathbf{w}^\top \phi(\mathbf{x})$. As a concrete example, when $\mathbf{x} = [x_1, x_2]$ then

$\phi(\mathbf{x}) = [1, x_1, x_2, x_1x_2, x_1^2, x_2^2]$. A linear model then looks like:

$$h(\mathbf{x}) = \mathbf{w}^\top \phi(\mathbf{x}) = w_0 + w_1x_1 + w_2x_2 + w_3x_1x_2 + w_4x_1^2 + w_5x_2^2$$

Which is exactly the form of a degree 2 polynomial in two dimensions. In other words, fitting a model $\mathbf{w}^\top \phi(\mathbf{x})$ using a degree 2 polynomial feature map is equivalent to fitting a degree 2 polynomial to the data!

In general, you might consider the more extreme degree p polynomial feature map. Instead of writing down this feature map explicitly, just imagine it as containing all possible products of sets of p features. In other words, take any combination of *up to* p features from your original data, and take the product of those features and add them to your feature map. In general, the number of features grows *exponentially* in p – think about each new feature as a binary string of length d with p 1s, where each 1 indicates whether to include that feature in the product or not.

Pros of this approach: 2^p gives you more features than there are electrons in the universe for even moderate values of p , and we know that linear models become extremely powerful in high dimensions. With enough new features, your linear model is guaranteed to be able to fit the training data perfectly, and we'll have to rely on regularization to not overfit!

Cons of this approach: If $\phi(\mathbf{x})$ has more entries than there are electrons in the universe, writing them all down on a computer will probably take a fair amount of storage and quite some time.

The **kernel trick** is a way to achieve the best of both worlds. As we'll see, we can use extremely flexible and powerful feature expansions without paying heavy computational or storage complexities.

3 Kernel trick

The kernel trick is based around the observation that if we're clever about how we choose $\phi(\mathbf{x})$, we might be able to compute inner products $\phi(\mathbf{x})^\top \phi(\mathbf{z})$ even if we can't write down either the feature expansion of $\mathbf{x}$ or the feature expansion of $\mathbf{z}$. We'll see how this helps us later. For now, to give an example of this, consider the degree two polynomial feature expansion again:

$$\phi(\mathbf{x}) = \begin{bmatrix} 1 \\ x_1 \\ \vdots \\ x_d \\ x_1x_2 \\ \vdots \\ x_{d-1}x_d \\ x_1^2 \\ \vdots \\ x_d^2 \end{bmatrix}$$

If we take two feature vectors $\mathbf{x}$ and $\mathbf{z}$ and take the inner product of these feature expansions we get:

$$\phi(\mathbf{x})^\top \phi(\mathbf{z}) = 1 + \sum_i x_i z_i + \sum_i \sum_j x_i x_j z_i z_j = 1 + \mathbf{x}^\top \mathbf{z} + (\mathbf{x}^\top \mathbf{z})^2$$

To see this, just apply the usual dot product formula to the expansion above. You end up with all pairs of terms represented, and then all pairs of pairs represented. For example, since x_i and z_j are features in their respective expansions, $x_i z_j$ will be a term in the inner product. Similarly, since $x_1 x_2$ is a feature in $\phi(\mathbf{x})$ and $z_5 z_6$ is a feature in $\phi(\mathbf{z})$, $x_1 x_2 z_5 z_6$ will be a term in the inner product – the second sum takes care of these.

Crucially however, the final expression – $1 + \mathbf{x}^\top \mathbf{z} + (\mathbf{x}^\top \mathbf{z})^2$ – can be computed from just knowing the original inner product $\mathbf{x}^\top \mathbf{z}$! In general, for a degree p polynomial feature expansion, **up to constants in some features** we can write the inner product of the implied feature expansion as:

$$k_{\text{poly}}(\mathbf{x}, \mathbf{z}) = (1 + \mathbf{x}^\top \mathbf{z})^p$$

Let's recap. The above function takes as input $\mathbf{x}$ and $\mathbf{z}$ and computes *in linear time* the value of the inner product you'd get if you first mapped $\mathbf{x}, \mathbf{z} \rightarrow \phi(\mathbf{x}), \phi(\mathbf{z})$ using a degree p polynomial feature expansion. That's a pretty impressive feat considering, if d and p are moderately large, we can't even write down $\phi(\mathbf{x})$ before the heat death of the universe!

Of course, all we've done is shown how to cheaply compute inner products in certain kinds of feature maps – we still need to show how to do machine learning. To do this, we make the following key fact.

Key Fact (Roughly the Representer Theorem)

Let $\mathbf{x}_1, \dots, \mathbf{x}_n$ be a set of training examples equipped with labels $y_1, \dots, y_n$. Any linear model $\mathbf{w}^*$ trained on this data can be written as a linear combination of the training examples:

$$\mathbf{w}^* = \sum_i \alpha_i \mathbf{x}_i$$

This is true regardless of the space that learning is done in. For example, if we first map $\mathbf{x}_i \rightarrow \phi(\mathbf{x}_i)$ using some feature map ϕ , a linear model trained in the new feature space can still be represented as:

$$\mathbf{w}^* = \sum_i \alpha_i \phi(\mathbf{x}_i)$$

How does this fact help us? Well, most of the loss functions we've seen have computed $\mathbf{w}^\top \mathbf{z}$ as part of the loss, and made predictions via $\mathbf{w}^\top \mathbf{z}$. Using the above fact, what we'll do is simply replace:

$$\mathbf{w}^\top \mathbf{z} \rightarrow \sum_i \alpha_i \mathbf{x}_i^\top \mathbf{z}$$

everywhere we see it both when computing losses and making predictions. Why is this helpful? Well, imagine we now use a feature map for all $\mathbf{x}_i$ as well as $\mathbf{z}$:

$$\mathbf{w}^\top \mathbf{z} \rightarrow \sum_i \alpha_i \underbrace{\phi(\mathbf{x}_i)^\top \phi(\mathbf{z})}_{=k(\mathbf{x}_i, \mathbf{z})}$$

Now, we only ever access data through **inner products after the feature map**. If we can find some kernel function that helps us compute $\phi(\mathbf{x})^\top \phi(\mathbf{z})$ very cheaply like we could for the polynomial feature map, then we can compute $\mathbf{w}^\top \mathbf{z}$ very cheaply!

$$\mathbf{w}^\top \mathbf{z} \rightarrow \sum_i \alpha_i k(\mathbf{x}_i, \mathbf{z})$$

4 Example application: kernel ridge regression

Recall that ridge regression minimizes the squared loss plus an L2 regularizer:

$$\min_{\mathbf{w}} \sum_{i=1}^n (\mathbf{w}^\top \mathbf{x}_i - y_i)^2 + \lambda \|\mathbf{w}\|_2^2 = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \lambda \mathbf{w}^\top \mathbf{w}$$

An aside. While the above key fact holds in general, it's actually very easy to show that the above key fact holds for the optimal model $\mathbf{w}^*$ learned by minimizing this objective. Suppose there is an optimal $\mathbf{w}^* = \mathbf{u} + \mathbf{v}$, where $\mathbf{u} = \sum_i \alpha_i \mathbf{x}_i$ is the component of $\mathbf{w}^*$ that **can** be written as a linear combination of the $\mathbf{x}_i$ vectors, but there's an additional component $\mathbf{v}$ that is orthogonal to each $\mathbf{x}_i$. Then:

$$\mathbf{w}^{*\top} \mathbf{x}_i = \mathbf{u}^\top \mathbf{x}_i + \underbrace{\mathbf{v}^\top \mathbf{x}_i}_{0 \text{ by definition}} = \mathbf{u}^\top \mathbf{x}_i$$

Since $\mathbf{w}^{*\top} \mathbf{x}_i = \mathbf{u}^\top \mathbf{x}_i$ for all i , clearly $\|\mathbf{X}\mathbf{w}^* - \mathbf{y}\|_2^2 = \|\mathbf{X}\mathbf{u} - \mathbf{y}\|_2^2$. However, $\|\mathbf{w}^*\|^2 = \|\mathbf{u}\|^2 + \|\mathbf{v}\|^2$, which means that $\|\mathbf{u}\|^2 < \|\mathbf{w}^*\|^2$. Thus $\mathbf{u}$ – which *is* a linear combination of the training features, obtains the same squared loss value but a lower L2 regularization value, so achieves a better overall objective than $\mathbf{w}^*$, which contradicts $\mathbf{w}^*$ being optimal.

The kernelized loss. With that out of the way, we can now apply the kernel trick to ridge regression. Using our replacement $\mathbf{w} \rightarrow \sum_j \alpha_j \mathbf{x}_j$, the ridge regression loss becomes:

$$\min_{\boldsymbol{\alpha}} \sum_{i=1}^n \left(\underbrace{\sum_{j=1}^n \alpha_j \mathbf{x}_j^\top \mathbf{x}_i}_{\mathbf{w} \in \mathbb{R}^d} - y_i \right)^2 + \lambda \underbrace{\sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j \mathbf{x}_i^\top \mathbf{x}_j}_{\mathbf{w}^\top \mathbf{w}}$$

Here, we are now minimizing over $\boldsymbol{\alpha}$ rather than $\mathbf{w}$ directly. Clearly, these are the same thing: if there exists a $\boldsymbol{\alpha}$ so that $\mathbf{w}^* = \sum_i \alpha_i \mathbf{x}_i$, clearly finding $\boldsymbol{\alpha}^*$ is as good as finding $\mathbf{w}^*$. Now, we apply our feature map $\mathbf{x} \rightarrow \phi(\mathbf{x})$ and do learning in that space instead:

$$\min_{\boldsymbol{\alpha}} \sum_{i=1}^n \left(\underbrace{\sum_{j=1}^n \alpha_j \phi(\mathbf{x}_j)^\top \phi(\mathbf{x}_i)}_{\mathbf{w} \in \mathbb{R}^D} - y_i \right)^2 + \lambda \underbrace{\sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)}_{\mathbf{w}^\top \mathbf{w}}$$

Finally, we apply the **kernel trick**, and replace the inner products $\phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j) = k(\mathbf{x}_i, \mathbf{x}_j)$ everywhere we see them.

$$\min_{\boldsymbol{\alpha}} \sum_{i=1}^n \left(\sum_{j=1}^n \alpha_j k(\mathbf{x}_j, \mathbf{x}_i) - y_i \right)^2 + \lambda \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j k(\mathbf{x}_i, \mathbf{x}_j)$$

All of our exponential time feature maps have been replaced by cheap to compute kernel functions!

The matrix version. A cleaner version of the loss can be derived using the matrix expansion $\mathbf{w} = \sum_i \alpha_i \mathbf{x}_i = \mathbf{X}^\top \boldsymbol{\alpha}$:

$$\begin{aligned} & \min_{\boldsymbol{\alpha}} \|\underbrace{\mathbf{X} \mathbf{X}^\top}_{\mathbf{w}} \boldsymbol{\alpha} - \mathbf{y}\|_2^2 + \lambda (\underbrace{\mathbf{X}^\top \boldsymbol{\alpha}}_{\mathbf{w}})^\top \underbrace{\mathbf{X}^\top \boldsymbol{\alpha}}_{\mathbf{w}} \\ &= \min_{\boldsymbol{\alpha}} \|\mathbf{X} \mathbf{X}^\top \boldsymbol{\alpha} - \mathbf{y}\|_2^2 + \lambda \boldsymbol{\alpha}^\top \mathbf{X} \mathbf{X}^\top \boldsymbol{\alpha} \end{aligned}$$

Observe that the matrix $\mathbf{X} \mathbf{X}^\top$ is just the inner product matrix, where $[\mathbf{X} \mathbf{X}^\top]_{ij} = \mathbf{x}_i^\top \mathbf{x}_j$. What we do is replace this with the **kernel matrix** $\mathbf{K}$ where $\mathbf{K}_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$:

$$\min_{\boldsymbol{\alpha}} \|\mathbf{K} \boldsymbol{\alpha} - \mathbf{y}\|_2^2 + \boldsymbol{\alpha}^\top \mathbf{K} \boldsymbol{\alpha}$$

Taking the derivative of the above with respect to $\boldsymbol{\alpha}$ and setting equal to zero, we solve:

$$\begin{aligned} & \nabla_{\boldsymbol{\alpha}} \|\mathbf{K} \boldsymbol{\alpha} - \mathbf{y}\|_2^2 + \boldsymbol{\alpha}^\top \mathbf{K} \boldsymbol{\alpha} \\ & \overbrace{2(\mathbf{K} \boldsymbol{\alpha} - \mathbf{y})^\top \mathbf{K} + 2\lambda \mathbf{K}} = 0 \\ & \mathbf{K}^2 \boldsymbol{\alpha} - \mathbf{K} \mathbf{y} + \lambda \mathbf{K} \boldsymbol{\alpha} = 0 \\ & \mathbf{K}^2 \boldsymbol{\alpha} + \lambda \mathbf{K} \boldsymbol{\alpha} = \mathbf{K} \mathbf{y} \\ & \mathbf{K}(\mathbf{K} + \lambda \mathbf{I}) \boldsymbol{\alpha} = \mathbf{K} \mathbf{y} \\ & (\mathbf{K} + \lambda \mathbf{I}) \boldsymbol{\alpha} = \mathbf{y} \\ & \boldsymbol{\alpha} = (\mathbf{K} + \lambda \mathbf{I})^{-1} \mathbf{y} \end{aligned}$$

Therefore, the optimal value of $\boldsymbol{\alpha}$ for kernel ridge regression is given by $(\mathbf{K} + \lambda \mathbf{I})^{-1} \mathbf{y}$. Put another way, the optimal linear model $\mathbf{w}^*$ in the feature space extracted by the feature map $\phi(\mathbf{x})$ corresponding to the kernel used is given by $\mathbf{w}^* = \mathbf{X}^\top \boldsymbol{\alpha}^*$.

Making predictions. After training a model like this, making predictions is pretty easy:

$$h(\mathbf{z}) = \mathbf{w}^{*\top} \phi(\mathbf{z}) = \sum_{i=1}^n \alpha_i^* \phi(\mathbf{x}_i)^\top \phi(\mathbf{z}) = \sum_{i=1}^n \alpha_i^* k(\mathbf{x}_i, \mathbf{z})$$

This is just a linear combination of easy to compute kernel values. For a matrix form, denote by $\mathbf{k}_{\mathbf{X}\mathbf{z}} = [k(\mathbf{x}_1, \mathbf{z}), \dots, k(\mathbf{x}_n, \mathbf{z})]$. Then:

$$h(\mathbf{z}) = \mathbf{k}_{\mathbf{X}\mathbf{z}}^\top \boldsymbol{\alpha}$$

Single step kernel ridge regression. Since we have a closed form solution for the optimal $\boldsymbol{\alpha}$, we can plug this in to get optimal kernel ridge regression predictions using the kernel $k(\cdot, \cdot)$:

$$h(\mathbf{z}) = \mathbf{k}_{\mathbf{X}\mathbf{z}}^\top (\mathbf{K} + \lambda \mathbf{I})^{-1} \mathbf{y}$$